Big Data

BUT Science des Données - Metz

Berhocoïrigoin Elena

Les informations contenues dans les slides avec des coins verts sont là pour votre connaissance personnelle, elles ne feront pas l'objet de questions pendant l' évaluation.

Chapitre 1. Traitement distribué : Spark

Chapitre 2. Spark Core & RDD

~~**Chapitre 3.** Spark DataFrames~~

Spark

- Développé en 2009 au sein de l'AMPLab de l'Université de Californie à Berkeley
- Spark peut être vu comme une amélioration de MapReduce

Spark

- Développé en 2009 au sein de l'AMPLab de l'Université de Californie à Berkeley
- Spark peut être vu comme une amélioration de MapReduce :
  - **Compatibilité avec Hadoop**
    HDFS, HBase, SequenceFiles etc.

Spark

- Développé en 2009 au sein de l'AMPLab de l'Université de Californie à Berkeley
- Spark peut être vu comme une amélioration de MapReduce :
  - **Compatibilité avec Hadoop**
  - **Calculs exécutés en mémoire** : rapidité de traitement
    10 à 100 fois plus rapide que Hadoop MapReduce

Spark

- Développé en 2009 au sein de l'AMPLab de l'Université de Californie à Berkeley
- Spark peut être vu comme une amélioration de MapReduce :
  - Compatibilité avec Hadoop
  - Calculs exécutés en mémoire
  - Cas d'usage supplémentaires
    requêtage SQL, machine learning, analyse de graphes, traitement de flux continu …

Spark

- Développé en 2009 au sein de l'AMPLab de l'Université de Californie à Berkeley
- Spark peut être vu comme une amélioration de MapReduce :
  - Compatibilité avec Hadoop
  - Calculs exécutés en mémoire
  - Cas d'usage supplémentaires
  - APIs haut-niveau multi-langage : accessibilité
    Scala, Java, Python, R

Spark et PySpark

Spark n'est pas une solution tout-en-un !
Il s'agit d'un logiciel de traitement de mégadonnées en environnement distribué, tout comme Hadoop MapReduce.

Système de fichier distribué

Base de données

Entrepôt de données

...

Spark et PySpark

Spark n'est pas une solution tout-en-un !
Pour fonctionner, il a besoin …

- d'une solution de stockage externe

Système de fichier distribué

Base de données

Entrepôt de données

…

Spark et PySpark

Spark n'est pas une solution tout-en-un !

Pour fonctionner, il a besoin …

- d'une solution de stockage externe

  exemple :

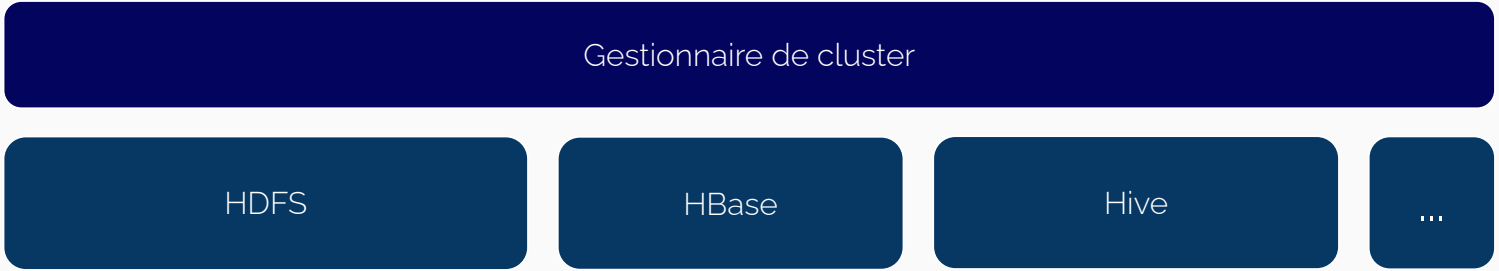
HDFS | HBase | Hive | …

Spark et PySpark

Spark n'est pas une solution tout-en-un !

Pour fonctionner, il a besoin …

- d'une solution de stockage externe
- d'un gestionnaire de cluster



Spark et PySpark

Spark n'est pas une solution tout-en-un !

Pour fonctionner, il a besoin …

- d'une solution de stockage externe
- d'un gestionnaire de cluster

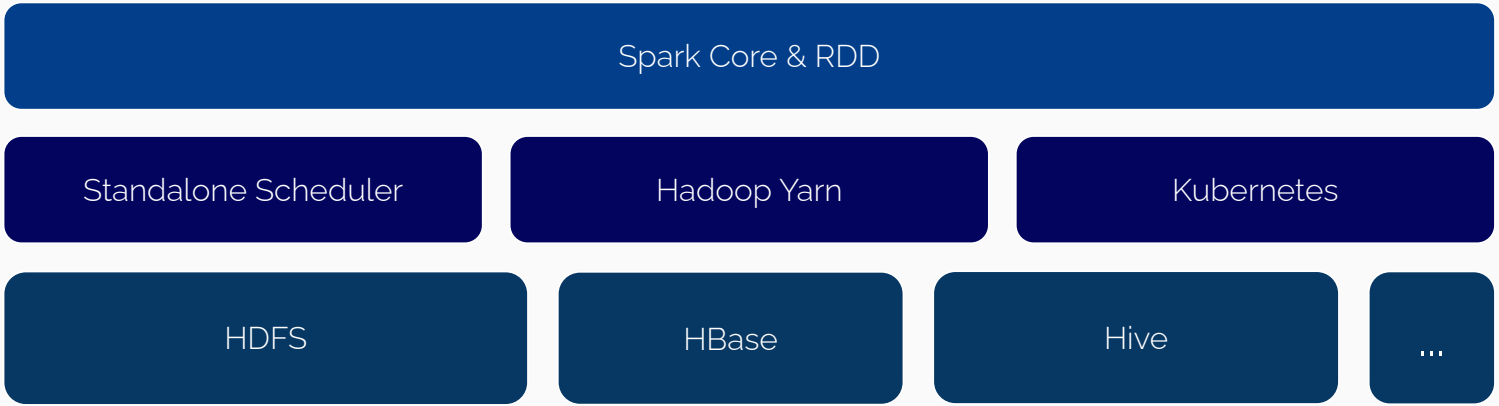
exemple :

Standalone Scheduler | Hadoop Yarn | Kubernetes

HDFS | HBase | Hive | …

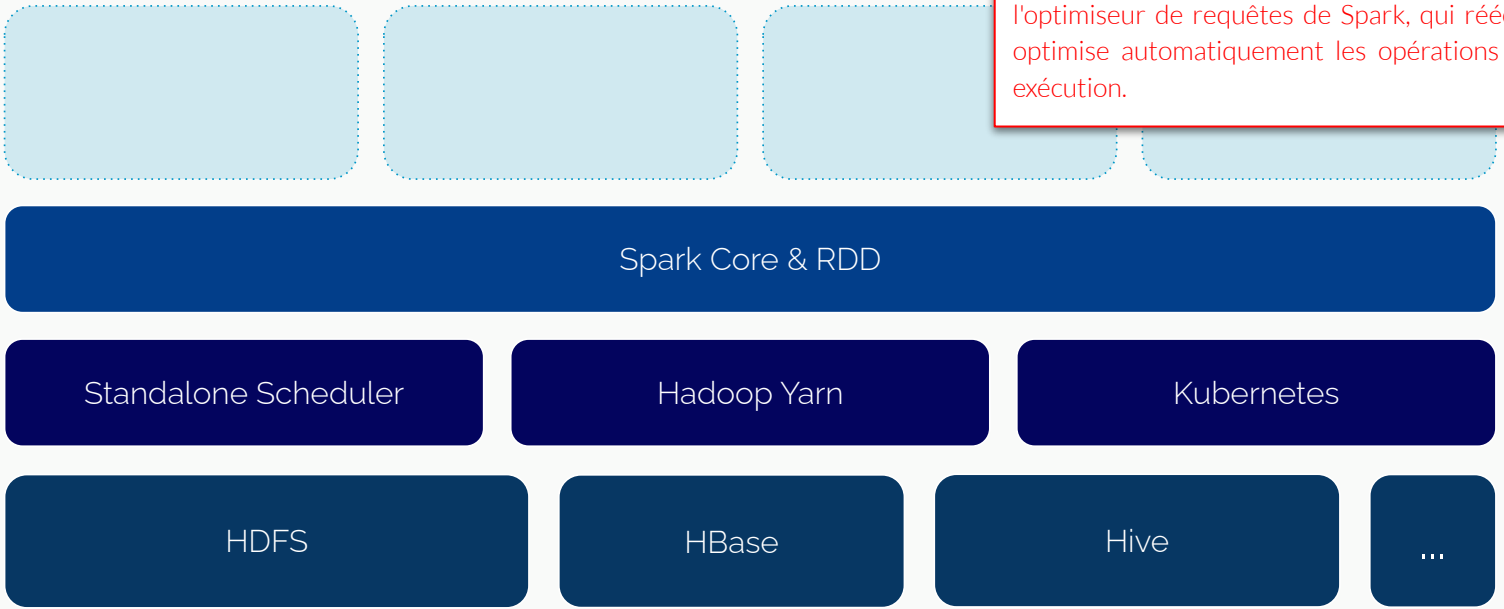
Spark et PySpark

Moteur d'exécution sur lequel repose toutes les autres fonctionnalités. Gestion de la mémoire, planification, allocation des tâches, tolérance aux pannes etc. Spark Core fournit la structure de données de base en Spark : les RDDs.



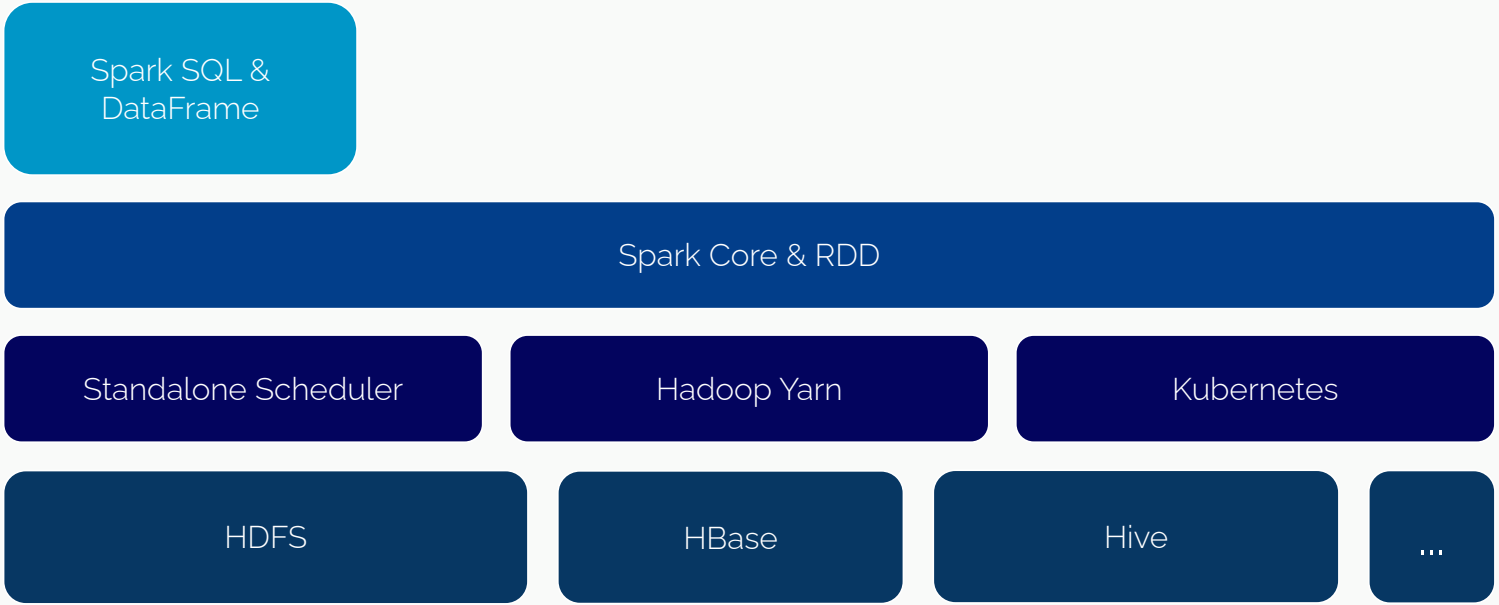
Spark et PySpark

Toutes les briques de cette couche facilitent l'utilisation de Spark. Utiliser ces outils de plus haut niveau permet également de bénéficier de l'optimiseur de requêtes de Spark, qui réécrit et optimise automatiquement les opérations avant exécution.



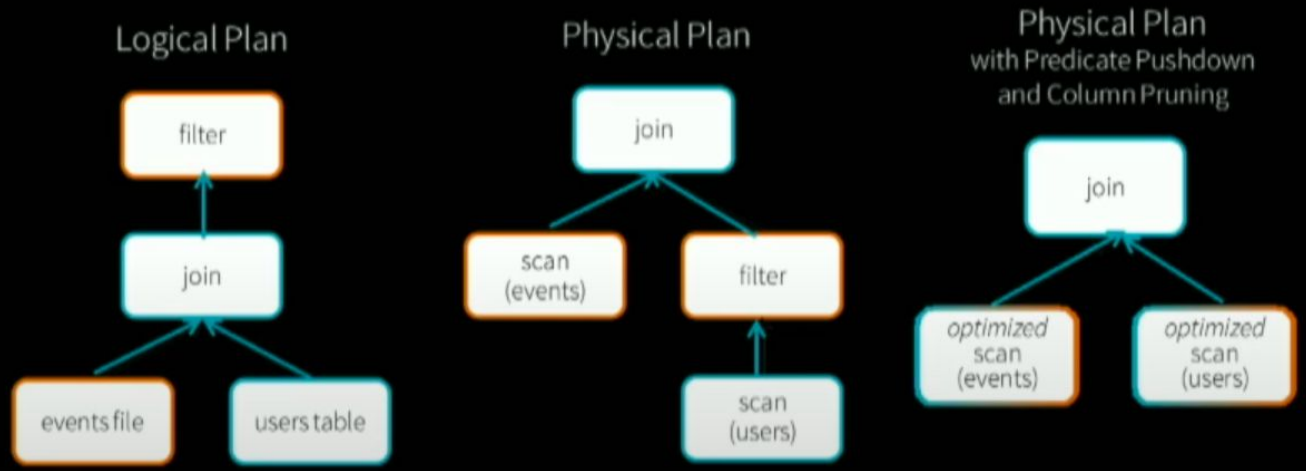
Spark et PySpark

Exécuter des requêtes SQL classiques.



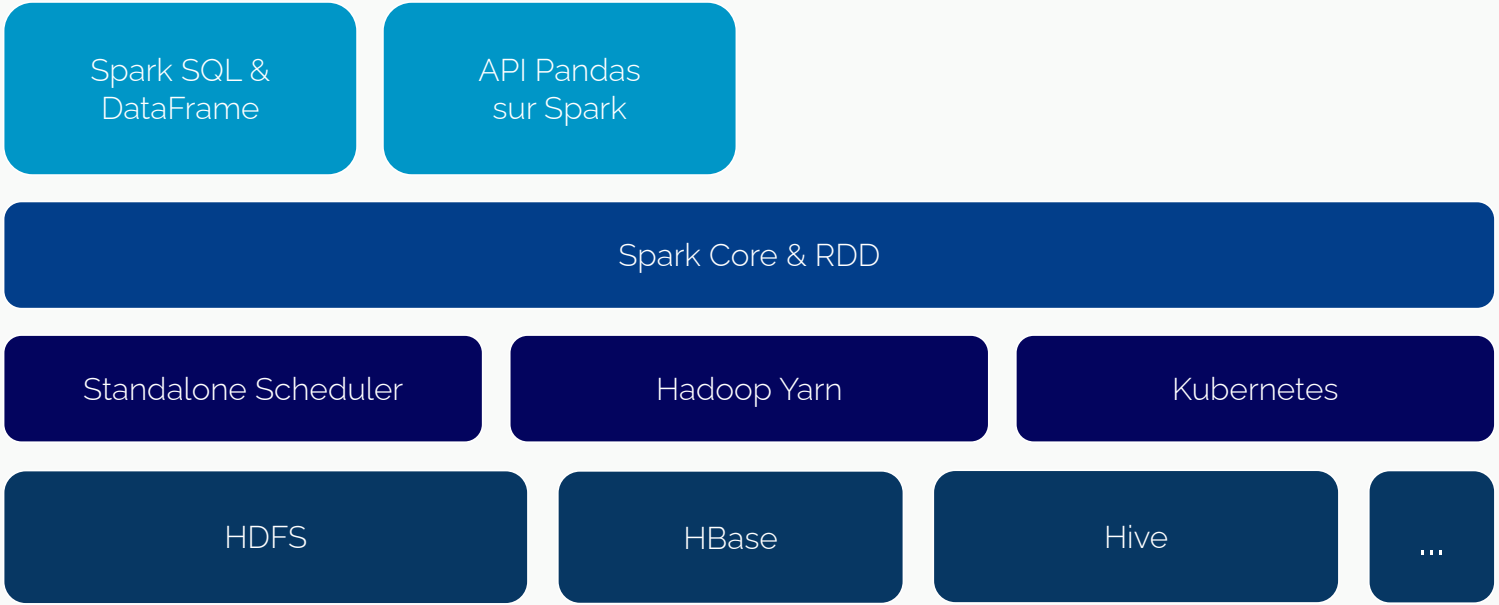
DataFrame Optimization

```
users.join(events, users("id") === events("uid")) .
filter(events("date") > "2015-01-01")
```



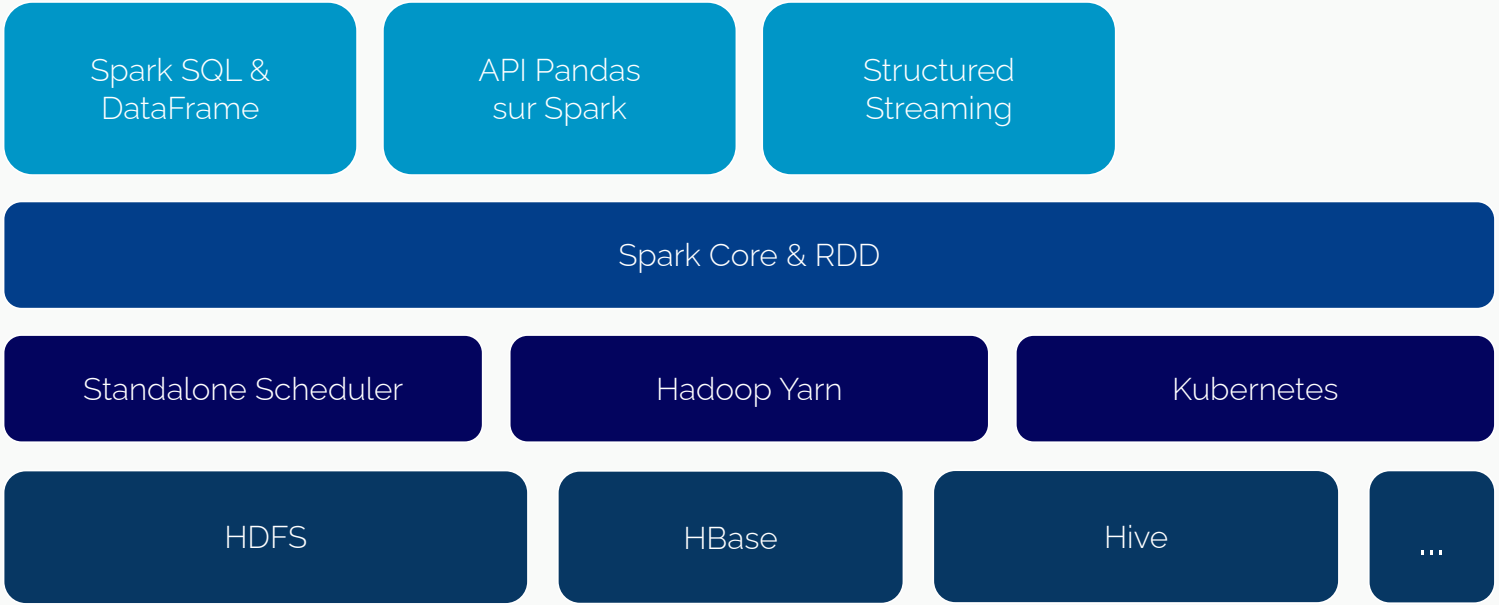
Spark et PySpark

Tirer profit de Spark via Pandas.



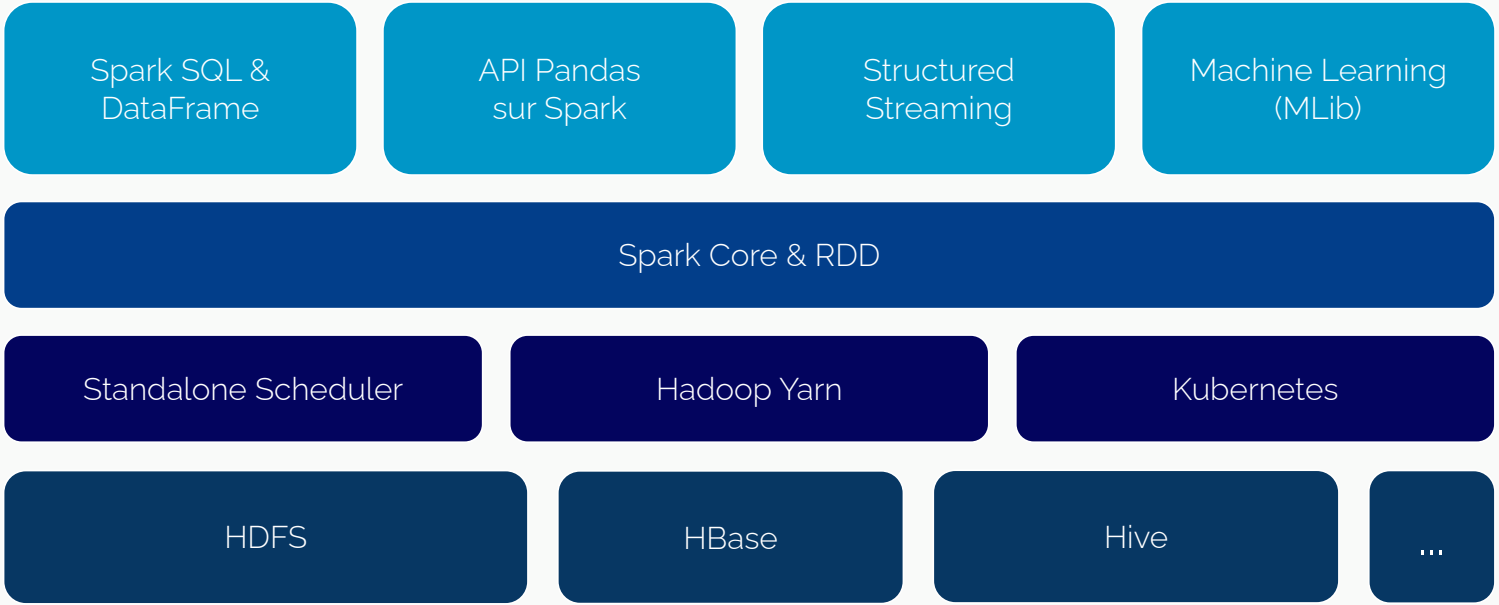
Spark et PySpark

Traiter des flux de données continus.

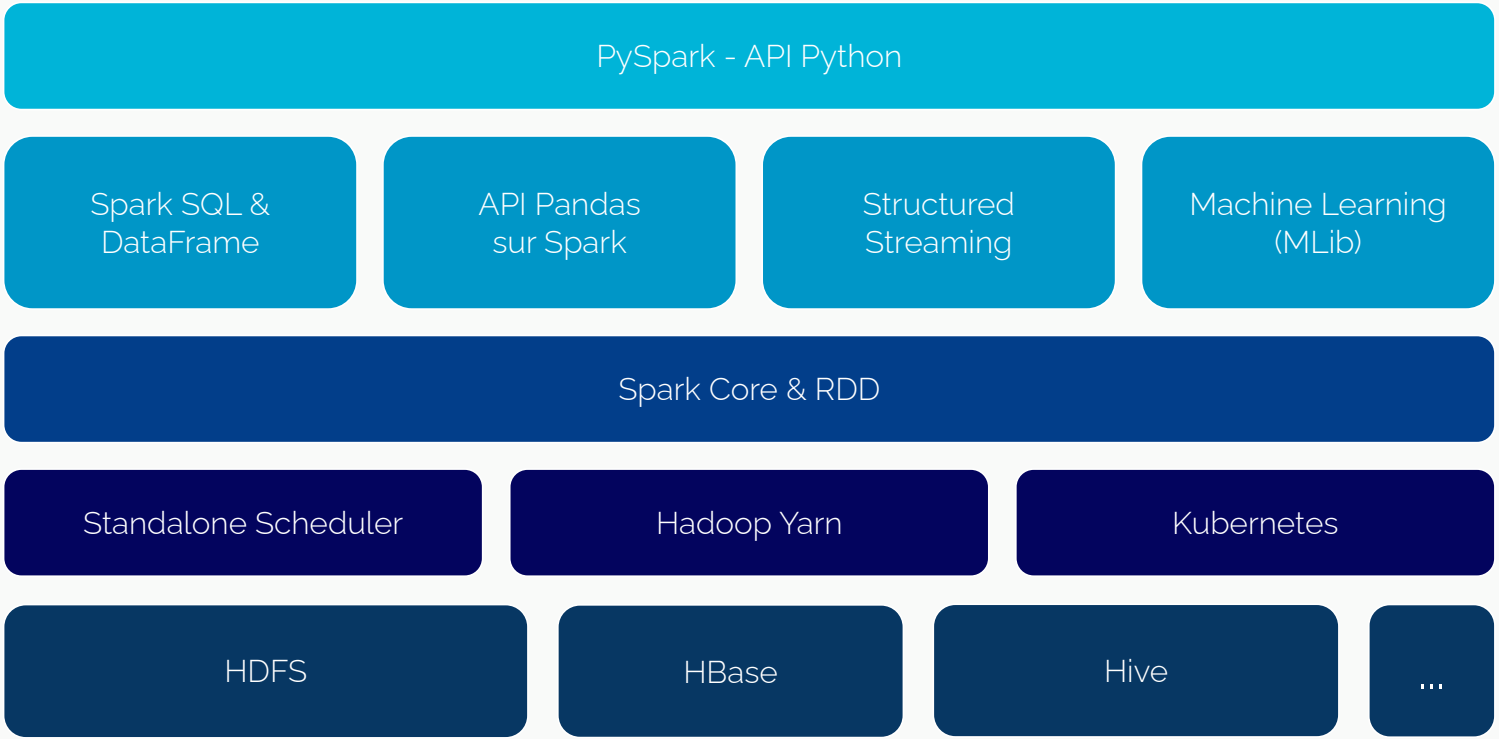


Spark et PySpark

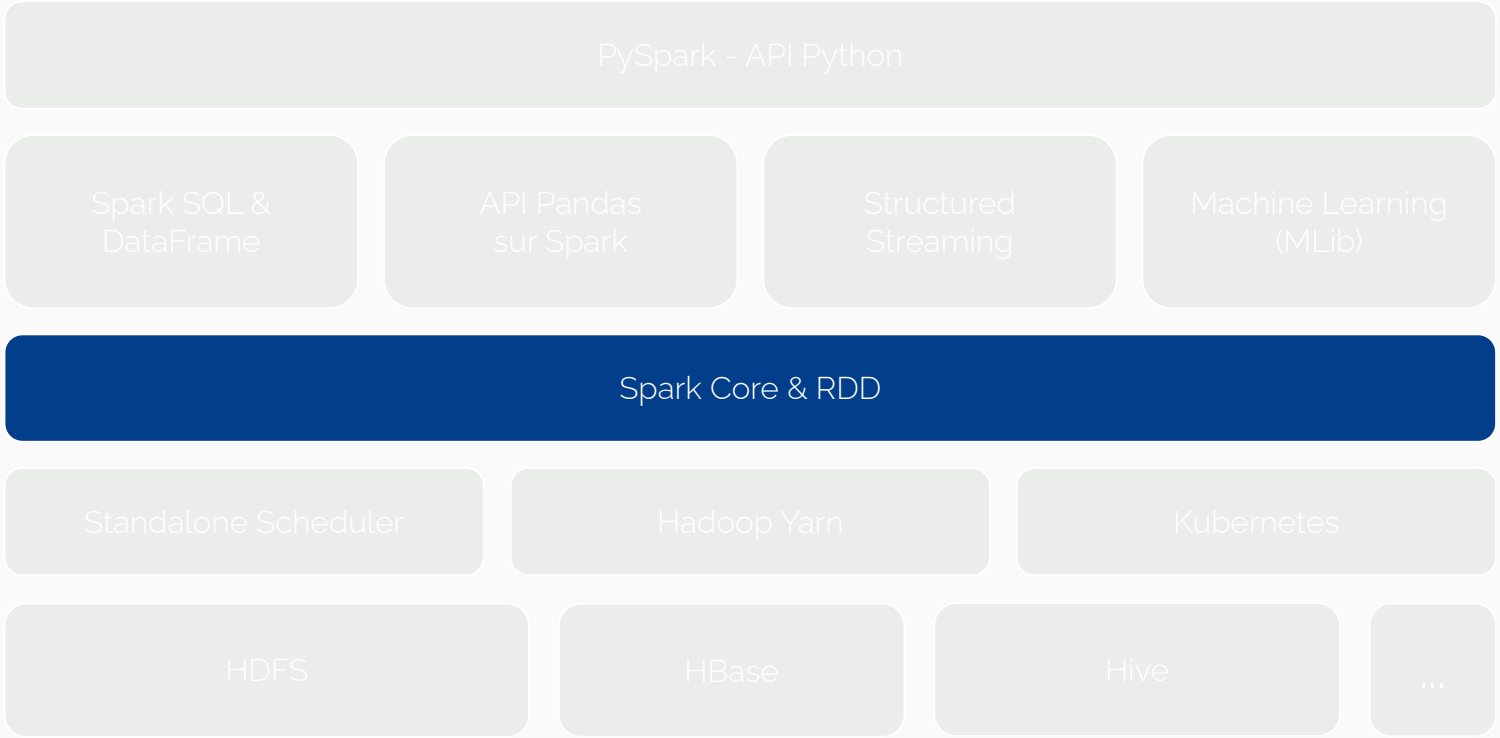
Bibliothèque d'apprentissage automatique.



Spark et PySpark



Spark et PySpark



Fonctionnement de Spark

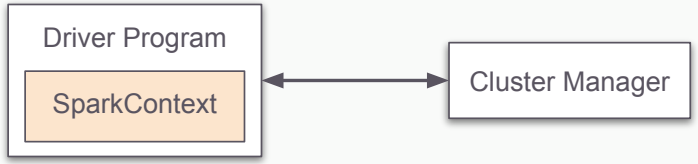
Pour exécuter un calcul distribué sur un cluster Spark …

1. Le programme principal crée un objet appelé **SparkContext**

Fonctionnement de Spark

Pour exécuter un calcul distribué sur un cluster Spark

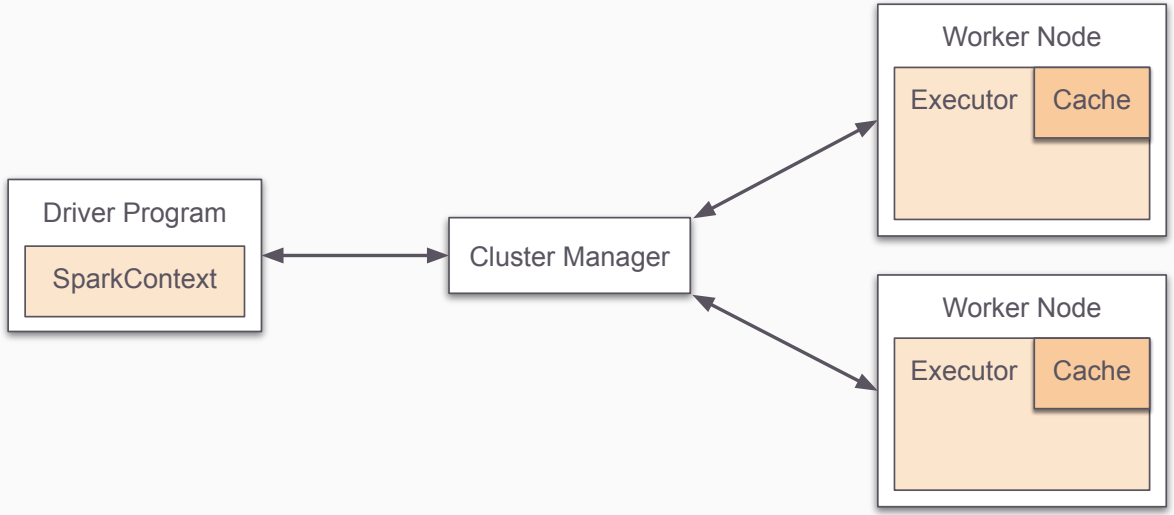
2. SparkContext se connecte à un **gestionnaire de cluster**



Fonctionnement de Spark

Pour exécuter un calcul distribué sur un cluster Spark

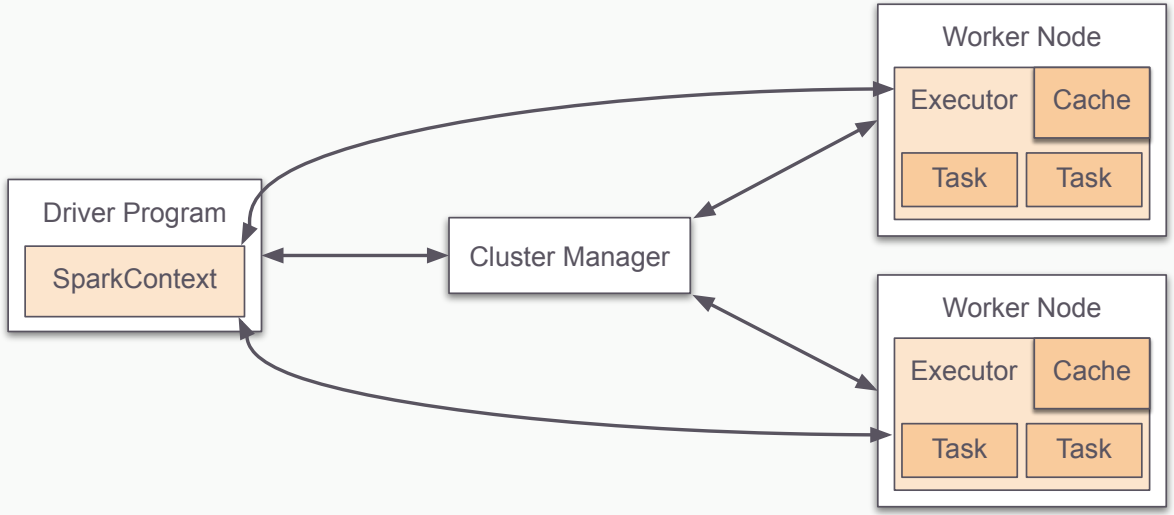
3. Une fois connecté, l'application Spark acquiert des **Executor** sur les noeuds du cluster (Worker Node)



Fonctionnement de Spark

Pour exécuter un calcul distribué sur un cluster Spark

4. SparkContext distribue le code et les **tâches** aux Executor pour qu'ils les exécutent en parallèle



Chapitre 1. Traitement distribué : Spark

Chapitre 2. Spark Core & RDD

Chapitre 3. Spark DataFrames

Spark Core & RDD

RDD (Resilient Distributed Dataset)

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** …

Spark Core & RDD

RDD (Resilient Distributed Dataset)

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** :

- **immuable**
  - un RDD ne peut pas être modifié
  - « modifier » un RDD crée un nouveau RDD
  - on conserve ainsi une trace des différentes opérations appliquées au RDD
  - on parle de **lignée** ou **lignage** (ou **lineage**)

Spark Core & RDD

RDD (Resilient Distributed Dataset)

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** :

- **immuable**
  - un RDD ne peut pas être modifié
  - « modifier » un RDD crée un nouveau RDD
  - on conserve ainsi une trace des différentes opérations appliquées au RDD
  - on parle de **lignée** ou **lignage** (ou **lineage**)

Spark Core & RDD

RDD (Resilient Distributed Dataset)

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** :

- **immuable**
- **partitionnée et distribuée**
  - un RDD peut être partitionné et distribué (partitionnement logique)

Spark Core & RDD

RDD (Resilient Distributed Dataset)

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** :

- **immuable**
- **partitionnée et distribuée**
- **qui peut être stockée en mémoire**
  - un RDD peut être mis en cache pour éviter les lectures / écritures disque répétées
  - par défaut, un RDD **n'est pas matérialisé**, ni en mémoire ni sur disque

Spark Core & RDD

RDD (Resilient Distributed Dataset)

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** :

- **immuable**
- **partitionnée et distribuée**
- **qui peut être stockée en mémoire**
- **tolérante aux pannes** (résiliente)
  - un RDD peut être reconstitué en cas de défaillance via le lineage (historique des opérations)

Reconstruction à la volée du RDD perdu en réappliquant les opérations nécessaires depuis **les données sources**. Pas besoin de répliquer les données pour la tolérance aux pannes.

Spark Core & RDD

RDD (Resilient Distributed Dataset)

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** :

- **immuable**
- **partitionnée et distribuée**
- **qui peut être stockée en mémoire**
- **tolérante aux pannes** (résiliente)
  - un RDD peut être reconstitué en cas de défaillance via le lineage (historique des opérations)

Reconstruction à la volée du RDD perdu en réappliquant les opérations nécessaires depuis **les données sources**. Pas besoin de répliquer les données pour la tolérance aux pannes.

Spark Core & RDD

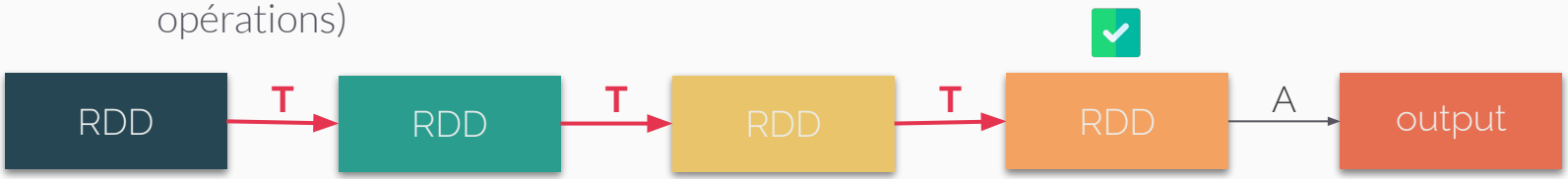
RDD (Resilient Distributed Dataset)

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** :

- **immuable**
- **partitionnée et distribuée**
- **qui peut être stockée en mémoire**
- **tolérante aux pannes** (résiliente)
  - un RDD peut être reconstitué en cas de défaillance via le lineage (historique des opérations)

Reconstruction à la volée du RDD perdu en réappliquant les opérations nécessaires depuis **les données sources**. Pas besoin de répliquer les données pour la tolérance aux pannes.



Spark Core & RDD

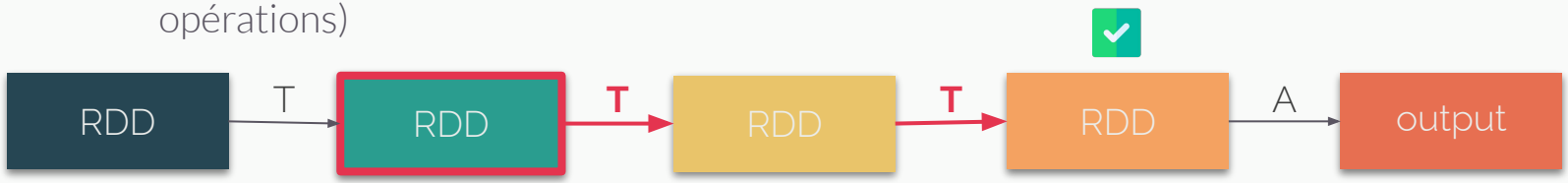
RDD (Resilient Distributed Dataset)

Reconstruction à la volée du RDD perdu en réappliquant les opérations nécessaires depuis un **RDD antérieur** stocké en mémoire ou sur disque.

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** :

- **immuable**
- **partitionnée et distribuée**
- **qui peut être stockée en mémoire**
- **tolérante aux pannes** (résiliente)
  - un RDD peut être reconstitué en cas de défaillance via le lineage (historique des opérations)



Spark Core & RDD

RDD (Resilient Distributed Dataset)

Brique de base et clé de la vitesse de traitement de Spark

Collection d'objets **structurés ou non structurés** :

- **immuable**
- **partitionnée et distribuée**
- **qui peut être stockée en mémoire**
- **tolérante aux pannes** (résiliente)
- que l'on peut **manipuler en mémoire** à l'aide de **divers opérateurs**

Opérations sur les RDD

Deux catégories d'opérations :

- Transformations
- Actions

Opérations sur les RDD

Transformation

- Une transformation **produit un nouveau RDD** à partir de l'ancien

Opérations sur les RDD

Transformation

- Une transformation **produit un nouveau RDD** à partir de l'ancien
- Les transformations sont « **paresseuses** » (lazy evaluation)

  Spark **n'exécute aucun calcul** tant qu'aucune Action n'est invoquée.

Opérations sur les RDD

Transformation

- Une transformation **produit un nouveau RDD** à partir de l'ancien
- Les transformations sont « **paresseuses** » (lazy evaluation)
- Deux catégories de transformation :
  - **Narrow transformation** (étroite)

    Une transformation étroite **ne nécessite pas de ré-agencer des données entre les nœuds du cluster** : chaque **partition** du RDD parent est utilisée par au plus une partition du RDD enfant (one-to-one)

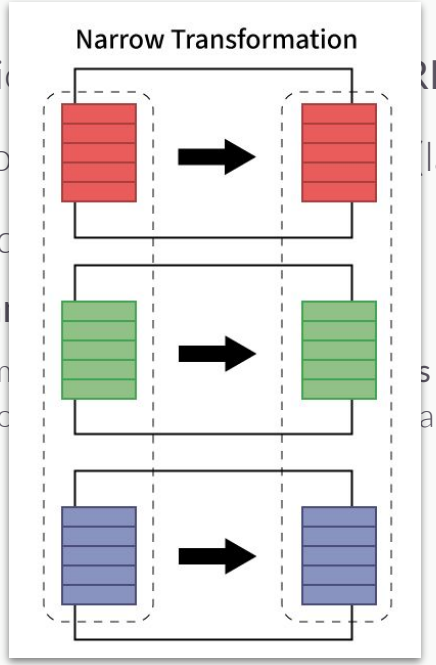
Opérations sur les RDD

Transformation

- Une transformati RDD à partir de l'ancien
- Les transformatio (lazy evaluation)
- Deux catégories c
  - Narrow tra

    Une transform s de ré-agencer des données entre les nœuds du cluster :
    chaque partitio ar au plus une partition du RDD enfant (one-to-one)

    exemple : ?



Opérations sur les RDD

Transformation

- Une transformation **produit un nouveau RDD** à partir de l'ancien
- Les transformations sont « **paresseuses** » (lazy evaluation)
- Deux catégories de transformation :
  - **Narrow transformation** (étroite)

    Une transformation étroite **ne nécessite pas de ré-agencer des données entre les nœuds du cluster** : chaque partition du RDD parent est utilisée par au plus une partition du RDD enfant (one-to-one)

    exemple : multiplier chaque nombre du jeu de données par 2

Opérations sur les RDD

Transformation

- Une transformation **produit un nouveau RDD** à partir de l'ancien
- Les transformations sont « **paresseuses** » (lazy evaluation)
- Deux catégories de transformation :
  - **Narrow transformation** (étroite)
  - **Wide transformation** (large)

    Une transformation large **nécessite de ré-agencer des données entre les nœuds du cluster** : plusieurs partitions du RDD enfant peuvent dépendre d'une même partition du RDD parent (one-to-many).

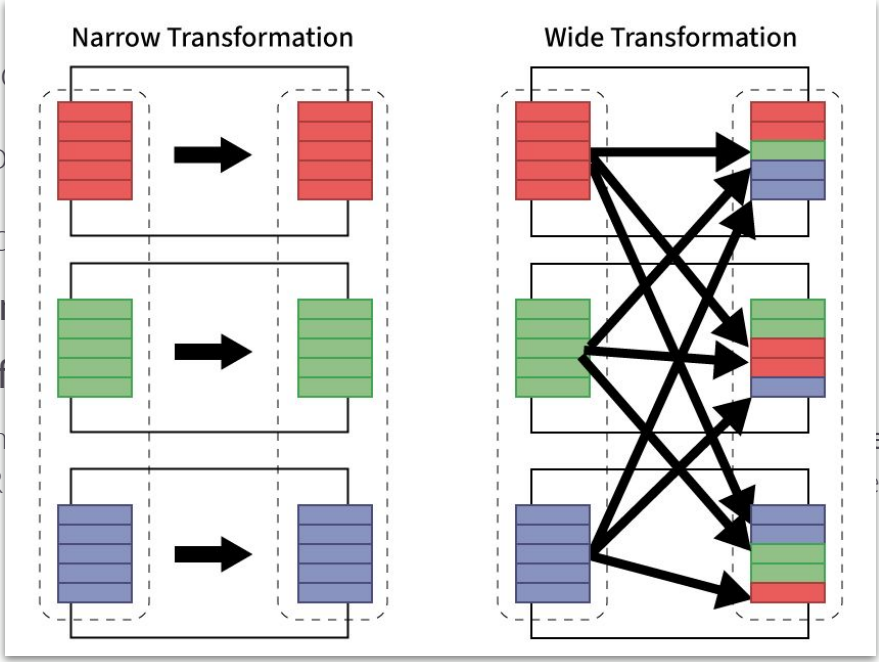
Opérations sur les RDD

Transformation

- Une transformati
- Les transformatio
- Deux catégories d
  - Narrow tra
  - Wide transf

    Une transform
    partitions du R

    exemple : ?



euds du cluster : plusieurs
nt (one-to-many).

Opérations sur les RDD

Transformation

- Une transformation **produit un nouveau RDD** à partir de l'ancien
- Les transformations sont « **paresseuses** » (lazy evaluation)
- Deux catégories de transformation :
  - **Narrow transformation** (étroite)
  - **Wide transformation** (large)

    Une transformation large **nécessite de ré-agencer des données entre les nœuds du cluster** : plusieurs partitions du RDD enfant peuvent dépendre d'une même partition du RDD parent (one-to-many).

    exemple : dédoublonner un jeu de données

Opérations sur les RDD

Transformation

- Une transformation **produit un nouveau RDD** à partir de l'ancien
- Les transformations sont « **paresseuses** » (lazy evaluation)
- Deux catégories de transformation :
  - **Narrow transformation** (étroite)
  - **Wide transformation** (large)

    Une transformation large **nécessite de ré-agencer des données entre les nœuds du cluster** : plusieurs partitions du RDD enfant peuvent dépendre d'une même partition du RDD parent (one-to-many).

    exemple : dédoublonner un jeu de données

    Partition n°1 : (A,1), (C,3)
    Partition n°2 : (B,2), (A,1), (B,2)
    Étape 1

    Partition n°1 : (A,1), (A,1), (C,3)
    Partition n°2 : (B,2), (B,2)
    Étape 2 (shuffle)

    Sortie : (A,1), (B,2), (C,3)
    Étape 3

Opérations sur les RDD

Transformation

Fonction	Description	Catégorie
`groupByKey()`	grouper les données par clé	wide
`filter()`	filtrer les données	narrow
`map()`	appliquer une fonction à chaque élément	narrow
`intersection()`	intersection de deux RDDs	narrow
`distinct()`	dédupliquer les données	
`union()`	union de deux RDDs	

Opérations sur les RDD

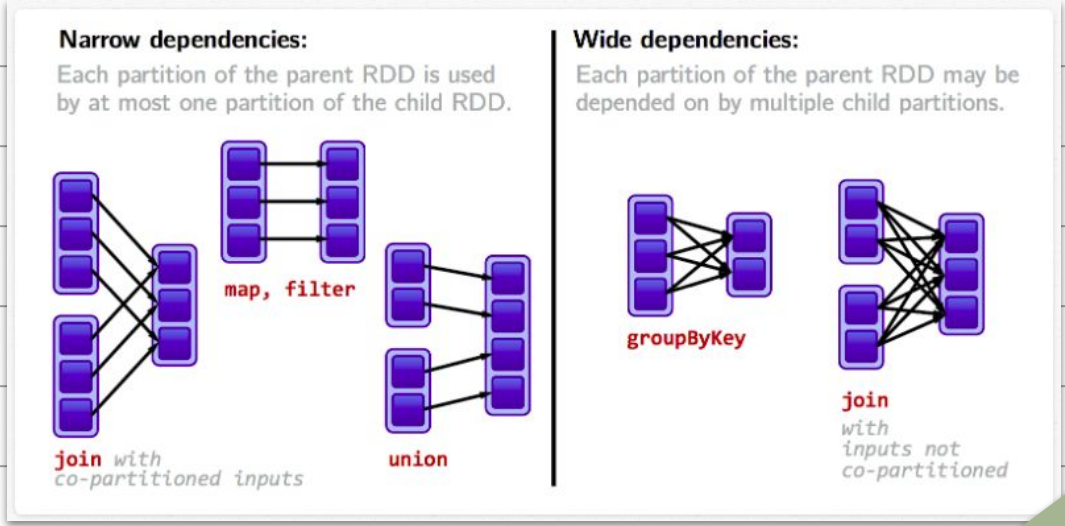
Transformation

Fonction	Description	Catégorie
`groupByKey()`	grouper les données par clé	wide
`filter()`	filtrer les données	narrow
`map()`	appliquer une fonction à chaque élément	narrow
`intersection()`	intersection de deux RDDs	wide
`distinct()`	dédupliquer les données	wide
`union()`	union de deux RDDs	narrow

Opérations sur les RDD

Transformation

Fonction	Description	Catégorie
`groupByKey()`		
`filter()`		
`map()`		
`intersection()`		
`distinct()`		
`union()`		



Opérations sur les RDD

Action

- Une action **produit une valeur** à partir d'un RDD, renvoyée au programme principal, ou **écrit des données** dans un système de stockage externe

  exemple : `count()`, `reduce()`, `collect()`

Opérations sur les RDD

Action

- Une action **produit une valeur** à partir d'un RDD, renvoyée au programme principal, ou **écrit des données** dans un système de stockage externe
- Les actions déclenchent l'exécution des calculs

  Tant qu'aucune Action n'est invoquée, les données ne sont même pas accédées.
  Seules les Actions peuvent matérialiser l'ensemble du processus avec des données réelles.

Opérations sur les RDD

Action

- Une action **produit une valeur** à partir d'un RDD, renvoyée au programme principal, ou **écrit des données** dans un système de stockage externe
- Les actions déclenchent l'exécution des calculs
- dont **Persistence** : mise en cache d'un RDD (option stockage RAM, disque, ou mixte)

  exemple : `persist()`, `cache()`

DAG, Stage & Task

DAG (Directed Acyclic Graphs)

- L'évaluation paresseuse permet de planifier l'exécution

DAG, Stage & Task

DAG (Directed Acyclic Graphs)

- L'évaluation paresseuse permet de planifier l'exécution
- Spark construit un DAG à partir des opérations spécifiées dans le code

DAG, Stage & Task

DAG (Directed Acyclic Graphs)

- L'évaluation paresseuse permet de planifier l'exécution
- Spark construit un DAG à partir des opérations spécifiées dans le code
- Graphe orienté sans cycle où :
  - Chaque noeud représente un RDD intermédiaire
  - Chaque arête correspond à une transformation ou une action

DAG, Stage & Task

DAG (Directed Acyclic Graphs)

- L'évaluation paresseuse permet de planifier l'exécution
- Spark construit un DAG à partir des opérations spécifiées dans le code
- Graphe orienté sans cycle où :
  - Chaque noeud représente un RDD intermédiaire
  - Chaque arête correspond à une transformation ou une action
- Le DAG est organisé en étapes (**stages**), composées de tâches (**tasks**) qui peuvent être exécutées en parallèle
  - Les stages sont délimitées par des shuffles

    Une étape est donc une succession de …

DAG, Stage & Task

DAG (Directed Acyclic Graphs)

- L'évaluation paresseuse permet de planifier l'exécution
- Spark construit un DAG à partir des opérations spécifiées dans le code
- Graphe orienté sans cycle où :
  - Chaque noeud représente un RDD intermédiaire
  - Chaque arête correspond à une transformation ou une action
- Le DAG est organisé en étapes (**stages**), composées de tâches (**tasks**) qui peuvent être exécutées en parallèle
  - Les stages sont délimitées par des shuffles
    Une étape est donc une succession de narrow transformation

DAG, Stage & Task

DAG (Directed Acyclic Graphs)

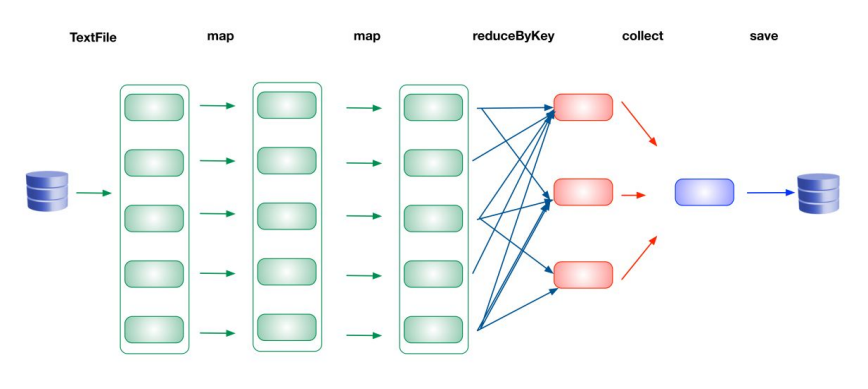
- L'évaluation paresseuse permet de planifier l'exécution
- Spark construit un DAG à partir des opérations spécifiées dans le code
- Graphe orienté sans cycle où :
  - Chaque noeud représente un RDD intermédiaire
  - Chaque arête correspond à une transformation ou une action
- Le DAG est organisé en étapes (**stages**), composées de tâches (**tasks**) qui peuvent être exécutées en parallèle
- Le DAG permet une optimisation globale du plan d'exécution
  Décomposition en étapes, réordonnancement des tâches, suppression des opérations inutiles etc.
  D'autant plus si une API Spark de haut niveau est utilisée

Aperçu

```
text_file = sc.textFile(''hdfs:/...'')
counts = text_file.flatMap( lambda line : line.split( " " ) )
                  .map( lambda word : ( word , 1 ) )
                  .reduceByKey( lambda a , b : a + b )
output = counts.collect()
output.saveAsTextFile(''hdfs://...'')
```

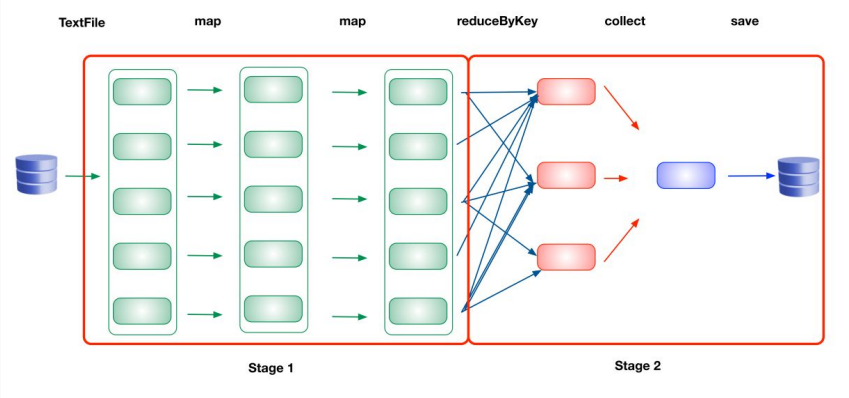
Aperçu

```
text_file = sc.textFile(''hdfs:/...'')
counts = text_file.flatMap( lambda line : line.split( " " ) )
                  .map( lambda word : ( word , 1 ) )
                  .reduceByKey( lambda a , b : a + b )
output = counts.collect()
output.saveAsTextFile(''hdfs://...'')
```



Aperçu

```
text_file = sc.textFile(''hdfs:/...'')
counts = text_file.flatMap( lambda line : line.split( " " ) )
                  .map( lambda word : ( word , 1 ) )
                  .reduceByKey( lambda a , b : a + b )
output = counts.collect()
output.saveAsTextFile(''hdfs://...'')
```



MERCI